[Early Bird Access] TikTok TechJam 2026 Tracks & Problem Statements

Congratulations on gaining exclusive Early Bird access to the TikTok TechJam 2026 Problem Statements ahead of the public release! *Not sure how to or where to start? Want to learn more from our Problem Statement setting engineers?*

Join our **technical workshop webinars** to learn more about the tracks that interest you! Look forward to key takeaways about the problems, our technologies, and the impact your projects can bring. You are highly encouraged to attend all the workshops that align with your interests, so as to make a more informed decision in selecting the track that works best for you.

- **Date**: 28 August 2026, Friday (SGT/GMT+8)
- **Time**: 1-6pm (see below for the actual 45min time slot of each track)
- **Webinar Link:** **https://vc-my.larkoffice.com/j/484622806**

Webinar Schedule (28 Aug)

The link to the webinars will be shared after the Public release of Problem Statements. *Stay tuned!*

Date	Time	Track
28 Aug (Fri)	1:00pm - 1:45pm	Track 1: **Agent Launchpad: Design and Build Lightweight Agent Middleware**
	2:00pm - 2:45pm	Track 2: **Autonomous Machine Learning Research Agent for Recommender Systems**
	3:00pm - 3:45pm	Track 3: **Implement a GPU Kernel for a Transformer Layer**
	4:00pm - 4:45pm	Track 4: **Shopping Copilot: AI Conversational Search and Recommendations**
	5:00pm - 5:45pm	Track 5: **Robust Detection of AI-Generated Images Under Real-World Transformations**

1. Agent Launchpad: Design and Build Lightweight Agent Middleware

Technical Workshop Webinar with Q&A will be held on **28 Aug, 1:00 to 1:45pm.**

Click here to join the webinar!

Build the missing middleware, not the platform.

The **Starter Kit** already provides the browser UI, Agent CRUD, Playground, backend control plane, persistent workspaces, Codex CLI Runtime, BytePlus ModelArk integration, local containers, and optional ECS deployment. Identity and authorization, trace and audit, layered Agent architecture, and threat modeling and safety are recommended middleware examples—not a prescribed checklist. Teams may choose, combine, simplify, replace, or invent capabilities that make the Agent platform more usable, manageable, observable, secure, reliable, or extensible.

https://github.com/RrankPyramid/CodeJam

Starter Kit — RrankPyramid/CodeJam

1.1 Challenge Overview

AI Agents are software actors that can reason, call tools, execute code, read and write files, and continue work across multiple turns. A useful Agent platform therefore needs more than a chat box: operators must be able to understand what happened, control what an Agent may access, and contain unsafe execution.

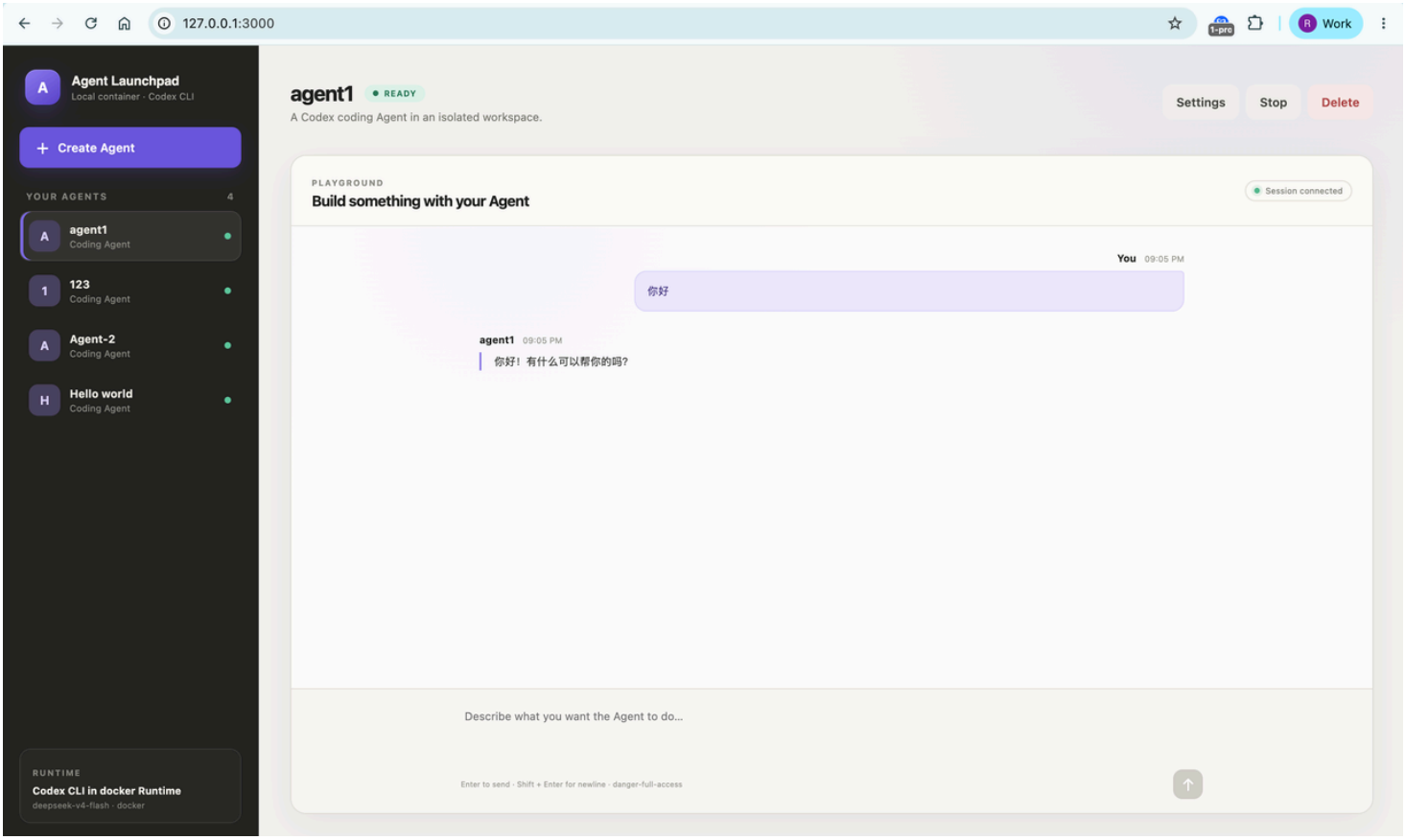
Building the full web application, control plane, cloud deployment, model connection, and Agent Runtime from scratch would consume the entire hackathon. This challenge removes that bottleneck. Every team starts from the same working platform and spends the three days on one meaningful infrastructure problem.

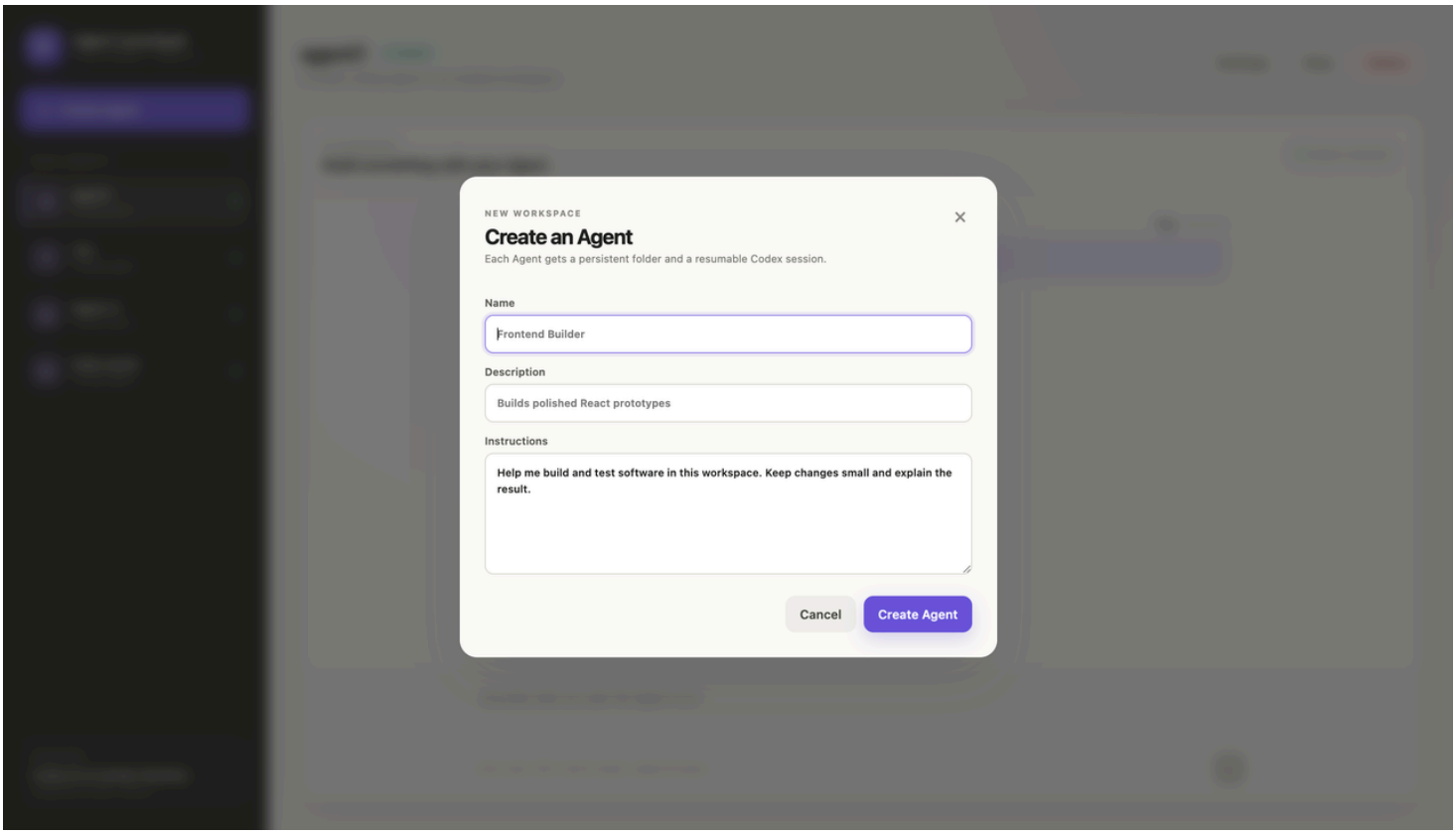
Your goal: design and demonstrate a coherent middleware story that improves the Agent platform in a functional, testable way without breaking the provided lifecycle or Playground. Evaluation focuses on the relevance, quality, and integration of the capabilities your team chooses or invents.

Area	Provided by the Starter Kit	Student responsibility
Product experience	React UI, Agent list, Create/Edit forms, lifecycle controls, Playground, Run status.	Keep the baseline working; add only the UI needed to expose your middleware.
Control plane	Fastify API, validation, asynchronous Runs, AgentService, JSON persistence.	Integrate real middleware behavior into the backend path.
Agent Runtime	Codex CLI, persistent sessions, per-Agent workspaces, disposable local containers.	Integrate team-designed middleware at the most appropriate execution boundary.
Infrastructure	Docker, Colima, Podman, Docker Compose, ECS scripts, and Terraform.	Use the smallest runtime path that proves your design. Cloud deployment is optional.
Middleware	Intentionally absent: no user identity, trace timeline, audit model, or hardened sandbox policy.	Select, adapt, combine, or invent a coherent set of middleware capabilities and demonstrate why they improve the platform.

1.2 Starter Kit

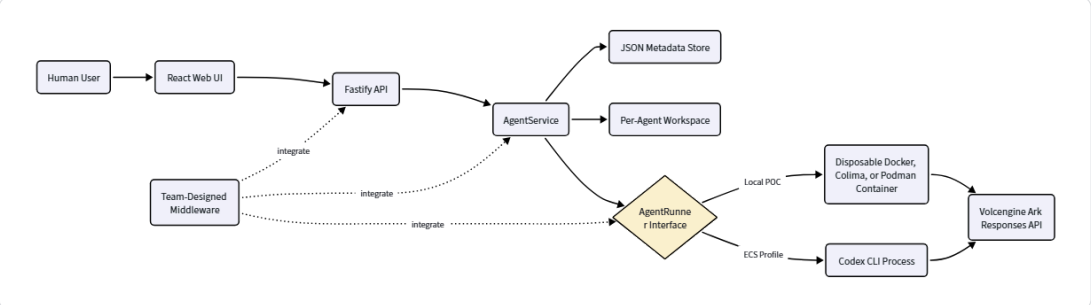
What already works





- Create, inspect, edit, start, stop, and delete Agents from the browser.
- Send multi-turn tasks through the Playground and poll asynchronous Run status.
- Let Codex CLI write files and run commands inside the selected Agent workspace.
- Resume the same Codex session in later messages.
- Persist Agent, message, and Run metadata in a local JSON store.
- Run each local turn in a disposable Docker, Colima, or Podman container.
- Connect Codex to a BytePlus ModelArk Responses-compatible endpoint.
- Deploy the same POC to an existing BytePlus ECS instance or provision an ECS environment with Terraform.

Current architecture and extension points



The Fastify request boundary, `AgentService`, the `AgentRunner` interface, and the execution data model are all valid extension seams. The diagram deliberately uses one generic Team-Designed Middleware node: teams may add new events, principals, policies, lifecycle behavior, provider adapters, memory controls, reliability mechanisms, or other capabilities wherever their design has the strongest boundary.

Profile	Agent execution	Use during the hackathon
Local POC	One disposable local container per turn.	Recommended development and judging path. Supports Docker, Colima, and rootless Podman.

BytePlus ECS	Codex runs inside the application container.	Optional deployment path for teams that want a cloud demo.
Local development	Codex runs as a host process.	Useful for hot reload when the host Codex CLI is installed and configured.

Intentional limitations

The repository is a single-user POC. Its optional bearer token protects a remote demo but is not a user identity or authorization system. The JSON store supports one process. Ordinary containers are not a hardened multi-tenant isolation boundary. These limitations are deliberate extension points, not hidden requirements to fix all at once.

1.3 Run the Baseline Locally

Requirements

- macOS or Linux.
- Node.js 22 or newer and npm 10 or newer.
- One container engine: Docker, Colima, or Podman.
- A BytePlus ModelArk API key and a Responses-compatible endpoint ID.

Clone and start

Clone the Starter Kit

```
git clone https://github.com/RrankPyramid/CodeJam.git
cd CodeJam
```

One-command local POC

```
ARK_API_KEY=your-ark-api-key ARK_MODEL=ep-your-endpoint-id npm run poc
```

The first run installs Node.js dependencies and builds the Runtime image. The startup script automatically selects Docker, Colima, or Podman. Open http://localhost:3000 when the server is ready.

Ark credential requirement: `ARK_API_KEY` must be an Ark model API key, not a BytePlus account AK/SK. `ARK_MODEL` is normally an endpoint ID beginning with `ep-`. A wrong credential produces a `401 Unauthorized` response from the Ark Responses API.

Force rootless Podman

```
CONTAINER_ENGINE=podman ARK_API_KEY=your-ark-api-key ARK_MODEL=ep-your-endpoint-
```

Colima exposes the Docker CLI, so use the normal command after running `colima start`. Press `Ctrl+C` to stop the POC. Agent workspaces and conversations remain available for the next run.

Baseline acceptance test

1. Open the browser and select **Create Agent**.
2. Enter a name, description, and workspace instructions.
3. Create the Agent and send the following task in the Playground.

Example Playground task

```
Create a TypeScript hello-world CLI, add a test, run it, and summarize the files
```

1. Wait for the Run to complete and confirm that an assistant response appears.
2. Send a follow-up message and confirm that the same Codex session continues.

3. Stop and restart the Agent, then confirm that the workspace still exists.

Do not start middleware work until this flow succeeds. If the baseline fails, check `docker info` or `podman info`, inspect `http://localhost:3000/api/system`, and verify the Ark key and endpoint.

Development and validation

The simplest workflow is to edit the code, stop the POC, and rerun `npm run poc`. Before submitting, run the repository validation suite:

Required repository check

```
npm run check
```

This command runs TypeScript checks, server tests, and production builds. Additional setup paths are documented in the repository:

- README and browser SOP
- Docker, Colima, and rootless Podman guide
- Architecture and extension seams
- Optional ECS deployment guide

1.4 Platform and Middleware Design Requirements

The Starter Kit defines the basic platform experience. Beyond that baseline, middleware design is the core of the challenge. Teams should identify an Agent-specific problem, decide which responsibilities belong in the frontend, control plane, Runtime, data layer, or infrastructure boundary, and implement the smallest coherent solution that proves the idea.

The directions in Section 7 are examples rather than mandatory requirements. Teams may choose one, combine related ideas, simplify an example, replace it, or invent a different capability. Breadth is not the goal: reviewers will reward a clear problem, thoughtful architecture, real integration, and convincing evidence.

- **Preserve the baseline:** Agent CRUD, lifecycle actions, Playground chat, persistence, and model execution must continue to work.
- **Implement real behavior:** the middleware must execute in a backend, Runtime, data, or infrastructure path. Static screens and hard-coded success messages do not qualify.
- **Define the boundary:** explain which component owns the decision or event, what data crosses the boundary, and what happens when it fails.
- **Demonstrate meaningful evidence:** show the normal behavior and an appropriate failure, denial, recovery, degraded, or abuse case for the team's design.
- **Add automated verification:** test the core middleware behavior rather than only rendering the UI.
- **Keep secrets out:** never commit or display API keys, AK/SK, passwords, bearer tokens, or unredacted sensitive payloads.
- **Prefer the smallest useful infrastructure:** local execution is the default judging path; ECS is optional and does not affect the score.

In scope	Out of scope
A coherent middleware story with one or more related capabilities, a real integration path, minimal UI, tests, and demo evidence.	Rebuilding the React app, CRUD API, Playground, Codex integration, container launcher, or a commercial cloud platform.
Mock users, protected fixtures, controlled failures, provider adapters, lifecycle controls, trace data, policy decisions, or reliability mechanisms.	Production OAuth, a general-purpose policy engine, a microVM runtime, a container scheduler, or multi-region infrastructure unless central to the team's idea.
Focused schema changes and refactors needed to make the middleware understandable and extensible.	Unrelated redesigns or cosmetic work that does not prove Agent infrastructure behavior.

1.5 Agent Lifecycle and Post-Creation Experience

The Starter Kit already provides a post-creation experience rather than ending at an unexplained success message. A user can find an Agent, inspect its status and configuration, start or stop it, use the Playground, review messages and Runs, continue a Codex session, and delete the Agent while following an explicit workspace archival policy.

Teams may extend this lifecycle when it supports their middleware design. The following actions are examples rather than mandatory checkpoints:

- Test or invoke the Agent with a sample input.
- Open the middleware evidence for a specific Run, such as a trace, audit decision, policy result, recovery record, or budget event.
- Distinguish human operations from Agent operations and introduce human approval where useful.
- Update configuration through a new version and show what changed.
- Rotate or revoke credentials, permissions, tools, or network access.
- Pause, resume, stop, retry, reconcile, or recover an Agent or Run.
- Delete the Agent and clean up or retain its state according to an explicit policy.

Teams should implement only the lifecycle behavior needed to make their chosen or invented middleware capability convincing. Rebuilding every lifecycle feature is not expected.

1.6 Possible Three-Day Implementation Plan

Day	Engineering goal	Exit evidence
1	Start and understand the baseline. Define the Agent-specific problem, choose or invent a coherent middleware story, specify the contract, and complete the first backend path.	The baseline passes; the team can trigger one real middleware behavior, event, decision, or control from an API or test.
2	Finish the core middleware path, persist its evidence, add the minimum UI, and implement the most important success and failure cases.	The complete scenario works end to end from the browser to the backend, Runtime, data, or infrastructure boundary.
3	Add automated tests, handle errors and cleanup, finish the architecture diagram and README, then rehearse the demo.	`npm run check` passes and the complete demonstration fits within three minutes.

1.7 Recommended Middleware Directions and Examples

Middleware design is a core part of this challenge. The examples below follow the same language and architecture boundaries as the original challenge brief. They are recommended examples—not a prescribed checklist. Teams may choose, combine, simplify, replace, or invent capabilities that better support their platform. Evaluation focuses on the relevance, quality, and integration of the capabilities the team designs.

Recommended Middleware Example: Identity and Authorization

Identity and authorization are one recommended middleware direction. Teams choosing this area may explore a distinction between a **human principal** and an **Agent principal**. For example, an Agent could use a separate identity instead of reusing a human session, personal access token, or shared platform credential.

Possible identity and authorization ideas include:

- **Human authentication:** identify the user who owns, creates, approves, updates, or stops an Agent.
- **Per-Agent identity:** create a distinct principal for an Agent or Agent version that can be rotated or revoked independently.
- **Delegated authority:** represent scoped, time-bound, and revocable permissions that describe what the Agent may access.
- **Policy enforcement:** perform authorization checks at a trusted backend, tool, data, or Runtime boundary rather than relying on UI restrictions.
- **Approval boundaries:** require optional human approval for selected external writes, high-cost actions, production operations, or sensitive data access.
- **Action attribution:** record the initiating human, executing Agent, requested scope, decision, target resource, and result.
- **Secret handling and revocation:** keep provider credentials on trusted backends, redact sensitive values, and demonstrate how later execution changes after revocation.

A small mock identity model is acceptable. For example, a team could show ownership isolation between User A and User B and prove that an Agent owned by User A cannot read User B's mock resource. A login screen without server-side authorization would not demonstrate the middleware itself.

Recommended Middleware Example: Trace, Audit, and Observability

Trace, audit, and observability are another recommended middleware direction. A team choosing this area could represent an Agent Run as a connected sequence of reasoning and actions rather than unrelated logs. Trace context may propagate across the frontend, control plane, Agent Runtime, model calls, tool calls, workspace operations, sandbox jobs, or cloud APIs that are relevant to the team's design.

Possible trace and audit ideas include:

- Stable identifiers such as Agent ID, Agent version, Run ID, session ID, trace ID, span ID, and actor type.
- Start time, duration, status, error details, and retry or cancellation relationships.
- Span categories such as orchestration, model call, tool call, memory access, sandbox execution, policy decision, human approval, or cloud operation.
- Inputs and outputs stored in a safely summarized or redacted form.
- Model, tool, Runtime, and infrastructure metadata needed to diagnose a Run.
- Token usage, cost, resource consumption, or other budget signals when available.

A trace-focused frontend could provide a Run list and a trace detail view with a tree or timeline, expandable spans, status filters, and a way to locate the failing step. A machine-readable query or export interface is an optional extension. Secrets and sensitive payloads should be redacted before storage or display.

Recommended Middleware Design Example: Layered Agent Architecture

Layered architecture is another recommended middleware design direction. Teams are encouraged to explain how responsibilities are organized, but no single layering model is required. The table below illustrates one possible architecture that can be simplified, adapted, or replaced.

Layer	Primary responsibility	Illustrative Starter Kit boundary
Experience Layer	Agent creation, catalog, Playground, middleware evidence, and lifecycle actions.	React Web UI calling stable platform APIs without holding the Ark key.
Control Plane	Agent specification, validation, status, Run orchestration, and reconciliation.	Fastify routes and `AgentService`.
Identity and Policy Plane	Human and Agent identity, delegation, approval, revocation, and audit.	A team-designed boundary around API, service, tool, or Runtime operations.
Agent Runtime Layer	Codex execution, model access, tool routing, retries, cancellation, and limits.	`AgentRunner`, local Runtime containers, or the ECS process.
Execution and Data Layer	Workspace files, persistent state, protected resources, connectors, and isolated execution.	Per-Agent workspaces, JSON metadata, mock services, or provider adapters.
Observability Layer	Trace ingestion, correlation, redaction, storage, query, visualization, and export.	New Run events, stores, APIs, and UI views added by the team.
Cloud Resource Layer	Compute, networking, storage, scheduling, and sandbox infrastructure.	Docker, Colima, Podman, or optional BytePlus ECS.

Teams may document the API or event contracts between selected layers and explain how their design could evolve to support another Runtime, identity provider, trace backend, tool, model, or infrastructure provider.

Recommended Middleware Example: Threat Modeling and Safety

Threat modeling and safety controls are another recommended middleware direction. Teams choosing this area could identify protected assets, actors, trust boundaries, abuse cases, implemented controls, and known residual risks. The table below provides examples; teams may focus on whichever threats are relevant to their design.

Threat	Controls to consider and demonstrate
Credential theft or exposure	Managed secret references, short-lived credentials, rotation, redaction, and exclusion of secrets from source, browser state, logs, and traces.
Privilege escalation or confused delegation	Least-privilege scopes, explicit delegation, backend policy checks, approvals, revocation, and complete actor attribution.

Prompt injection or tool misuse	Tool allowlists, typed schemas, target-resource scoping, output validation, execution limits, and approval for high-risk actions.
Sandbox escape or untrusted code	Non-privileged execution, restricted filesystems and networks, resource limits, controlled mounts, and patched Runtime images.
Cross-user access or data exfiltration	Ownership-aware authorization, storage isolation, scoped queries, outbound allowlists, protected metadata endpoints, and negative tests.
Runaway execution or cost	Timeouts, quotas, concurrency limits, maximum steps, token or cost budgets, and an administrative stop control.
Sensitive trace capture	Configurable capture levels, redaction before export, trace access control, and retention limits.

The Starter Kit's existing CPU, memory, PID, dropped-capability, and `no-new-privileges` defaults may be reused as baseline safeguards, but they do not by themselves constitute a new safety capability.

Recommended Middleware Example: Multi-Agent Coordination

Multi-Agent coordination is another recommended middleware direction. A team choosing this area may connect several Agent instances through a shared session, topic, queue, or lightweight coordinator. The purpose is not to build a complex distributed system; it is to demonstrate that the platform can route messages, preserve shared state, and coordinate turns across more than one Agent Runtime.

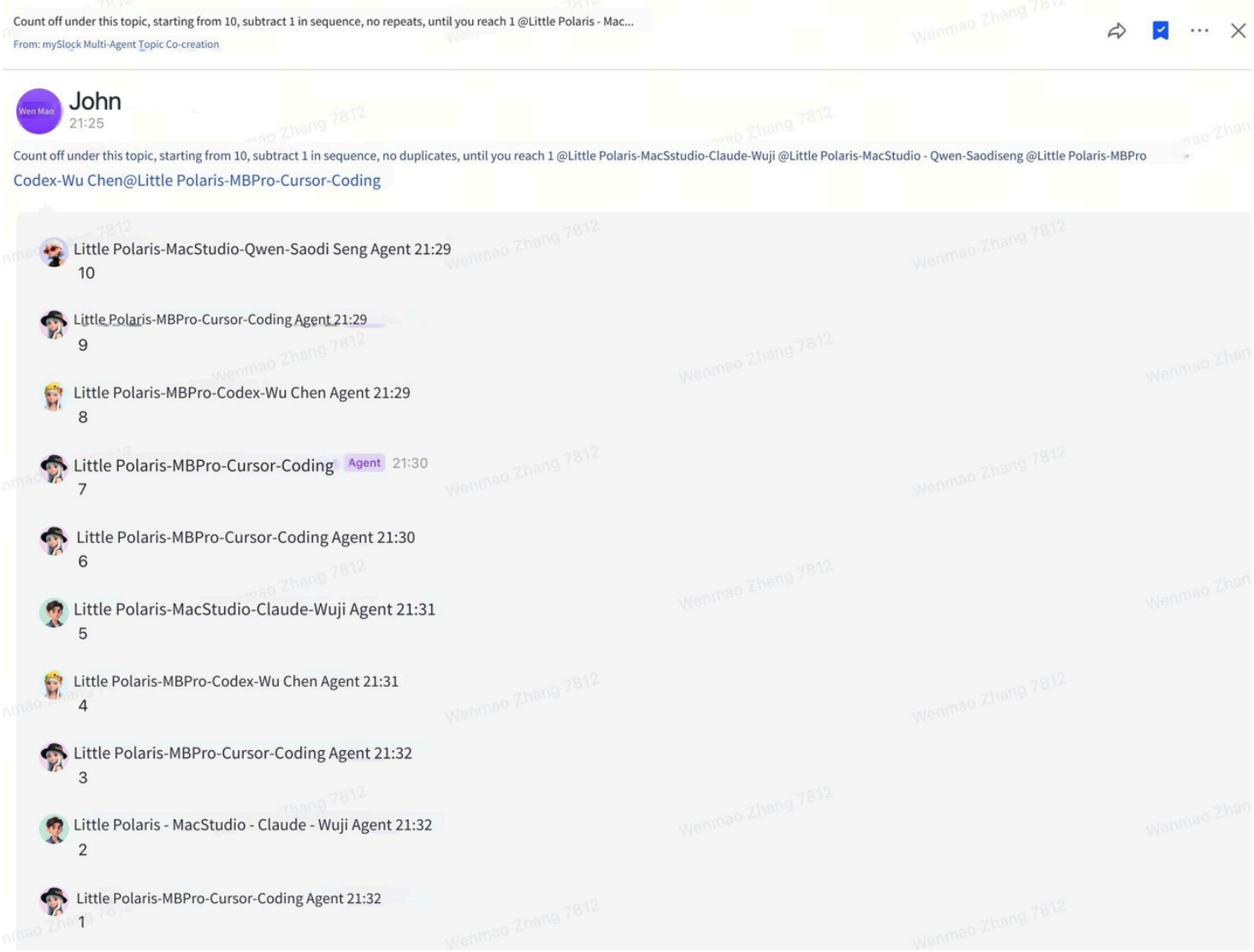
Example demo: create several Agents and ask them to count down from 10 to 1 in a shared conversation. On each turn, one Agent publishes the next unused number, then another Agent continues until the sequence reaches 1.

The minimum coordination layer may provide:

- A shared session or topic that all participating Agents can read and write.
- A simple turn-selection or message-routing rule.
- Shared state that records the latest number and prevents duplicate or skipped turns.
- A visible event history showing which Agent produced each number.
- A timeout, retry, or stop rule for an Agent that does not respond.

Successful demonstration: the team starts multiple Agents from the platform, launches one shared task, and shows a complete 10-to-1 sequence with no duplicate or missing number. The interface should make the participating Agent and ordering of each message clear. A platform-local webpage is sufficient; integration with an external chat product is optional.

Illustrative reference: multiple Agents take turns counting down in one shared topic.



Other Team-Designed Middleware

Teams are encouraged to propose capabilities outside the examples when they improve the Agent platform in a clear way. Possible directions include lifecycle reconciliation and failure recovery, state and memory governance, human-in-the-loop workflows, cost and budget control, provider abstraction, versioning and rollback, multi-Agent coordination, tool or model routing, credential exchange, or automated diagnosis and remediation.

A team-defined capability should still explain the Agent-specific problem, architecture boundary, functional evidence, failure or recovery case, and known limitations.

1.8 Required Live Demo

The live demo should show one complete scenario. The essential journey is a user creating or selecting a runnable Agent from the frontend and then using or testing it. Beyond that baseline, each team should demonstrate the middleware capabilities it chose or designed. Identity, trace, audit, policy, recovery, and revocation are examples rather than mandatory checkpoints.

1. Create or select an Agent from the frontend and show its current lifecycle state.
2. Invoke the Agent through the Playground with a real task.
3. Show at least one real model, file, tool, sandbox, data, or infrastructure action.
4. Demonstrate the middleware behavior and the evidence it produces.
5. Demonstrate an appropriate failure, denial, degraded, abuse, or recovery case.
6. Show that the platform remains understandable and controllable afterward.

The scenario may use a mock third-party service or controlled fixture. The frontend-to-Agent path and any middleware presented by the team must be functional rather than represented only by static screens.

1.9 Deliverables

1. **Three-minute live demo:** show one real Agent Run and the team-designed middleware working in its normal case and an appropriate failure, denial, recovery, degraded, or abuse case.
2. **One-page architecture diagram:** show the middleware, data flow, trust boundary, and enforcement, instrumentation, or recovery point.
3. **Code repository:** include setup instructions, the middleware problem and rationale, design summary, automated tests, demo steps, limitations, and no secrets.

1.10 Core Acceptance Checklist and Optional Evidence

- [ ] A reviewer can clone the repository, start the platform, and create or test an Agent from the frontend.
- [ ] The submission identifies and demonstrates one or more meaningful middleware capabilities selected, adapted, combined, or designed by the team.
- [ ] The middleware executes in a backend, Runtime, data, or infrastructure path rather than only in the UI.
- [ ] The repository and documentation are sufficient for reviewers to understand and reproduce the POC.
- [ ] `npm run check` passes.
- [ ] No secret appears in source, Git history, logs, traces, screenshots, browser storage, or demo output.
- [ ] Optional evidence: a delegated permission is scoped or revocable, enforced outside the UI, and demonstrated.
- [ ] Optional evidence: an end-to-end Agent Run produces a correlated trace with relevant model, tool, sandbox, policy, or infrastructure events.
- [ ] Optional evidence: a defined threat is blocked or contained, the protected asset remains unchanged, and cleanup or recovery is demonstrated.
- [ ] Optional evidence: a team-defined lifecycle, reliability, memory, budget, provider, or coordination capability works as described.

1.11 Evaluation Criteria

This track will follow the evaluation criteria below:

Category	Weight	What reviewers will look for
End-to-end middleware behavior	**40%**	A real frontend-to-backend, Runtime, data, or infrastructure path with convincing functional evidence.
Technical design and integration	**25%**	A clear rationale, coherent architecture, appropriate boundary, focused changes, and extensible contracts.
Verification and robustness	**20%**	Automated tests, error handling, cleanup or recovery, redaction, and protection against obvious bypasses.
Demo and reproducibility	**15%**	A concise live demo, useful README, one-command startup, documented limitations, and no hidden manual setup.

1.12 Scope Guidance and Frequently Asked Questions

This is a hackathon-scale Agent infrastructure challenge, not a requirement to build a complete commercial cloud product. A strong submission may support one local Runtime path, a small mock resource set, and a focused middleware story. Depth, coherence, and relevance matter more than the number of example features implemented.

Teams are not required to train a foundation model, build a workflow editor, implement production OAuth, create a general-purpose sandbox, support multiple cloud regions, or deploy to ECS. Mock external services are acceptable, but static UI mockups cannot replace functional middleware behavior.

Do we need BytePlus ECS? No. Local Docker, Colima, or Podman is the default judging path. Cloud deployment is optional.

Do we have to select one recommended example? No. The examples are starting points. Teams may adapt, combine, simplify, replace, or invent capabilities that fit their platform.

Can we use mock users or resources? Yes. Controlled fixtures are encouraged when they make middleware behavior reproducible.

Does a polished UI count as middleware? No. The UI may explain and visualize a capability, but the behavior must execute in a trusted backend, Runtime, data, or infrastructure path.

Why does Ark return 401 Unauthorized? The most common cause is using a BytePlus account AK/SK instead of an Ark model API key, or using the wrong endpoint ID.

Where should we start reading the code? Begin with `apps/server/src/types.ts`, `apps/server/src/app.ts`, `apps/server/src/agent-service.ts`, and the two `AgentRunner` implementations. Then inspect `apps/web/src/App.tsx` for the smallest UI integration point.

How to access the Starter Kit? https://github.com/RrankPyramid/CodeJam

2. Autonomous Machine Learning Research Agent for Recommender Systems

✨ **Technical Workshop Webinar with Q&A** will be held on **28 Aug, 2:00 to 2:45pm.**

Click here to join the webinar!

2.1 Background

Motivation

Machine learning engineers (MLEs) spend much of their time on a single activity: **taking a dataset and a set of metrics, then iterating on a model again and again to push the score higher.** This work is inherently cyclic — every round repeats the same loop, shown in Figure 1.

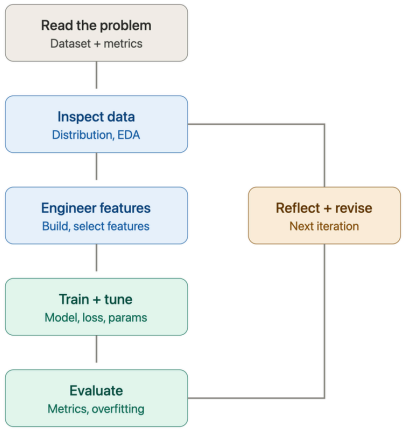


Figure 1. The MLE iteration loop

> **Figure 1. The MLE iteration loop.** A closed cycle of five core stages, plus a reflection step that feeds the next round:
>
> 1. **Read the problem** — understand the given dataset and the target metrics.
> 2. **Inspect data** — study data distribution through exploratory data analysis (EDA).
> 3. **Engineer features** — build and select input features (see Appendix A.5).
> 4. **Train + tune** — choose a model, set the loss function, and tune hyperparameters.
> 5. **Evaluate** — read the metrics, check for overfitting, and consult the leaderboard.
>
> The result of the **evaluate** stage drives a **reflect + revise** step, which decides what to change and loops back into the next iteration — re-inspecting the data and adjusting the features. The cycle repeats until the score plateaus.

Two of these stages — **engineer features** and **train + tune** — are carried out almost entirely in code: the engineer writes scripts to transform the data, define the model, and run training. In other words, each turn of the loop produces and modifies code. This is what makes the loop a natural target for automation: it is structured and repeatable, yet writing and revising that code is exactly the kind of task a code-generating LLM can take on.

The loop is also repetitive and mechanical. It draws heavily on "engineering intuition," but many individual steps are well-structured and repeatedly exercised in practice — which is precisely why automating the whole cycle has become an active research direction.

Prior Work

Over the past two years, a new line of work has set out to automate this loop: the **Autonomous ML Research Agent**, an LLM-driven agent that runs the cycle in Figure 1 on its own. It reads the problem, **writes the code** for each stage, trains and evaluates the model, reflects on the results, revises its approach, and finally produces a submission. Representative systems include:

- **MLE-Bench** [1] (OpenAI) — a benchmark of 75 Kaggle competitions, now a standard evaluation suite for such agents.
- **AIDE** [2] (Weco AI) — a state-of-the-art agent that frames ML engineering as code optimization and explores the space of solutions via tree search.

- **AI-Scientist-v2** [3] (Sakana AI) — an end-to-end agent for autonomous scientific and ML research, using agentic tree search to form hypotheses, run experiments, and write up results.

This Challenge

This challenge asks participants to design an **autonomous ML research agent**. Given a public ML dataset and a set of metrics, the agent must **autonomously** run the full loop of Figure 1 — read the problem, engineer features, train and tune the model, evaluate, then reflect and iterate — to reach the highest possible score across the test sets. Writing the code for each stage is part of the agent's job, not something provided in advance.

> **New to recommender systems?** Both benchmarks in this challenge come from the recommendation domain. If terms such as CTR, CVR, multi-task learning, or NDCG are unfamiliar, start with the **Appendix: A Primer on Recommender Systems** . At the end of this document — a concept map plus an annotated reading list designed to get you oriented in 1–2 hours.

2.2 Problem Statement

The Task

Design and implement an Autonomous ML Research Agent. For each benchmark, the agent must autonomously:

1. **Reproduce the official baseline.** Stand up a working end-to-end pipeline and confirm it reaches the official baseline's reported validation score. (The official baseline is a fixed, organizer-provided reference — see *Benchmarks*. Any starter pipeline the agent builds for itself is an internal step, not the reference it is scored against.)
2. **Iterate on the pipeline.** Autonomously draw on established methods from both industry and academia to improve each stage of the pipeline (see Figure 1), and apply those improvements in code. The agent develops using **only the training split and the public validation feedback** — it never has access to the hidden test set.
3. **Improve over the baseline.** Through repeated iterations, drive the **validation** score above the official baseline. Improvement need not be strictly monotonic — as with real-world data, the trajectory may fluctuate — but the agent should show a clear, sustained ability to keep improving relative to the baseline. Final ranking is computed once, on the **hidden test set**, using the submission the agent designates as final.

Task Requirements

1. **Runs end-to-end and aims to beat the baseline.** The agent must run the full pipeline on the required benchmark (AliCCP) and reach a converged result; attempting the bonus benchmark (KuaiRand) is optional. The target is a hidden-test score that exceeds the official baseline; the actual delta achieved — positive or negative — is what feeds into the Primary metric scoring (see Judging Criteria), so falling short of the baseline is scored continuously rather than treated as a disqualifying failure.
2. **Iterates autonomously across the full stack.** The agent should improve the solution on its own, driven by its own evaluation of results. Improvements may target any part of the algorithmic stack — not just the model architecture, but every upstream and downstream module is fair game. The goal is to **minimize human intervention** — a fully autonomous run is the ideal, but a well-instrumented **semi-automated** pipeline that requires only a handful of interventions is an acceptable and realistic outcome; in practice, we measure how little human intervention a run requires (e.g. the number of manual interventions).
3. **Robust operation.** The pipeline should run reliably with **minimal human intervention**. Robustness here is about how the agent *handles* difficulty, not how often it succeeds — we do not score it by failure count, since a capable agent may fail only on genuinely hard problems. What matters is that when a step fails (a code error, a timeout, an unexpected input), the agent can recover, retry, or route around it, and that long iterative runs neither crash, stall, nor diverge.

2.3 Constraints & Scope

Category	Constraints & Scope Details
In scope	• Any open-source library or framework (PyTorch, RecBole, TorchRec, LightGBM, …) • Any papers, public solutions, or pretrained weights • Changes to any pipeline stage — not just the model
Out of scope	• No external training data or pretrained weights trained on these benchmarks' test labels • No hidden-test access during development (train + validation only)
Limits	• **AliCCP**: CTR AUC (all impressions) / CVR AUC (clicked subset) (fixed) • **KuaiRand**: NDCG@10 / Recall@50, click = positive (fixed) (*Bonus*) • Hidden test scored once, on the final submission

	• Compute budget: *TBD*
Allowed assumptions	• Fixed `train / validation / hidden-test` split per dataset • Official baseline, scores & evaluation script (incl. convergence rule) • Example submission + output schema

2.4 Available Resources & Data

Starter Kit

To lower the barrier to entry — especially for participants new to recommender systems — the challenge provides a standard starting point:

1. **Fixed data splits**: Use the training / test split from NISE for AliCCP and you may need to split a validation set from the training set. Kuairand itself provides data splits according to dates (`log_standard_4_08_to_4_21_*.csv` & `log_standard_4_22_to_5_08_*.csv`) you can use 4/08–4/21 standard → train, 4/22–5/08 standard first 50% → validation, 4/22–5/08 standard last 50% → test. Teams develop on train + validation only, and evaluate the performance on test set.
2. **Official baseline**: a fixed, organizer-provided reference pipeline per dataset (refer to NISE for AliCCP and CWM for Kuairand), with its baseline scores published. Beating *this* baseline is what counts — not a baseline the team builds itself.
3. **Evaluation script**: the exact scoring code (CTR AUC over all impressions and CVR AUC over the clicked subset for AliCCP; NDCG@K / Recall@K for KuaiRand), plus the convergence rule (ε and *N*) and the absolute-delta aggregation. Refer to NISE for AliCCP, and CWM for Kuairand.
4. **Submission format**: a minimal, runnable example submission and the required output schema.
5. **Run-log requirements**: each iteration should record its **hypothesis**, the **code diff**, the resulting **metrics**, and any **error / recovery events**. These logs are how judges assess **Autonomy** (scored under Impact & Relevance) and **Robustness** (scored under Technical Execution) — see Judging Criteria.
6. **LLM coding agent**: you can use whatever you like, or use Trae from ByteDance, which provides "Limited offer: new user 7-day free trial".

Benchmarks

AliCCP is required and determines 100% of the primary score. **KuaiRand is a bonus dataset** — attempting it is optional and earns extra credit, but is not required to complete the primary score.

Resource policy. This is a hackathon, so external resources are open by default: use any open-source library (PyTorch, RecBole, TorchRec, LightGBM, …), read any papers, docs, or public solutions, and use pretrained model weights freely. The agent is expected to draw on whatever published methods it can find — that is what makes it a *research* agent.

There is **one hard rule: no external training data.** Training must rely only on these two datasets — no augmenting, joining, or pre-training on any other dataset, and no pretrained model whose weights were trained on these benchmarks' test labels. This single rule is what keeps the hidden-test ranking fair; everything else is unrestricted.

Dataset	Domain & Description	Metrics	Scale
AliCCP (Alibaba)	Taobao e-commerce. A CTR × CVR funnel: impression → click → conversion. A widely used benchmark for CTR/CVR and multi-task conversion modeling (ESMM / MMoE / PLE). **Evaluation space is fixed:** CTR AUC is computed over **all impressions** (positive = click); CVR AUC is computed over the **clicked subset** (positive = conversion), i.e. CVR = P(conversion | click). Teams may model CTCVR internally, but the two reported AUCs use these two spaces.	CTR AUC / CVR AUC	~80 million samples
KuaiRand (Kuaishou) (*Bonus dataset*)	Short-video feed. 12 feedback signals (click / like / follow / comment / forward / long_view / play_time …) plus a randomized-exposure intervention that supports counterfactual evaluation. **Relevance label and K are fixed by**	NDCG@K / Recall@K	~tens of millions of interactions

	the organizers (see Starter Kit / TBD): the default task treats `click` as the positive relevance label and reports **NDCG@10 / Recall@50**. The exact label definition and K values are pinned in the Starter Kit so every team solves the same task.		

Links: AliCCP — https://tianchi.aliyun.com/dataset/408; KuaiRand — https://kuairand.com

> KuaiRand's randomized-exposure data also enables off-policy / counterfactual evaluation (OPE). This is an **optional, advanced** direction and is not part of the primary score — see Appendix A.6.

2.5 Deliverables

1. **Written Project Description (via Devpost)**

- Provide a clear written description of your project that includes:
  - How your solution addresses the problem statement
  - Development tools used (e.g. VSCode, Colab, Jupyter)
  - APIs used (e.g. OpenAI GPT-4o, Google Maps API)
  - Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
  - Datasets and assets used (e.g. Google Local Reviews dataset, manually labelled data)

2. **Public Code/GitHub Repository**

- Submit a link to a public Code/GitHub repository containing:
  - Well-structured, commented code covering all components of your solution
  - A README file that includes:
    - Project overview
    - Setup and installation instructions
    - Steps to reproduce your results
    - A brief reflection on your solution's limitations and what you would improve given more time
    - Team member contributions (if applicable, i.e. team participants, non-solo participants)

3. **Run & Iteration Logs**

- Submit the per-iteration log required in the Starter Kit (Run-log requirements), covering:
  - Hypothesis for that iteration — what the agent intended to try and why
  - The code diff applied
  - The resulting metrics (NDCG@K / Recall@K for KuaiRand; CTR AUC / CVR AUC for AliCCP)
  - Any error or recovery events encountered, and how the agent handled them
- A short summary reporting the number of manual interventions during the run (used to assess autonomy per Task Requirement 2)

4. **Final Submission & Results Summary**

- Submit your final model output/checkpoint for the required benchmark (AliCCP), in the schema defined by the Starter Kit. If you also attempt the bonus benchmark (KuaiRand), submit its output as well for bonus scoring:
  - AliCCP (required) — final predictions/checkpoint used for the CTR AUC / CVR AUC evaluation
  - KuaiRand (bonus) — final predictions/checkpoint used for the NDCG@10 / Recall@50 evaluation
- A results table reporting your validation-best score for the required benchmark's metrics (AliCCP CTR AUC / CVR AUC), and its absolute delta over the official baseline (per the Evaluation section scoring formula); if you attempted the bonus benchmark (KuaiRand), include its NDCG@10 / Recall@50 results as well
- Reported resource usage required to reach the converged result: total token consumption (input + output) from the agent's LLM calls, and total GPU time (GPU-hours) consumed during training and evaluation (used to score Feasibility & Practicality)

2.6 Judging Criteria

Judging Criteria	Weight
Technical Execution	**35%**
Innovation & Problem Insight	**20%**
Impact & Relevance	**20%**
Feasibility & Practicality	**15%**
Presentation & Communication Final Event Only	**10%**

Technical Execution — Primary Metric & Robustness

Primary metric. We score the **converged result**, not the peak and not the intermediate trajectory. A run is considered converged when **validation score has not improved by more than a small threshold ε over the last *N* consecutive iterations** (default: ε and *N* fixed by the organizers and published in the Starter Kit), *or* when the run hits the fixed compute/wall-clock budget — whichever comes first. The submission scored for ranking is the **validation-best checkpoint** at that point, evaluated **once on the hidden test set**. The agent develops only on train + validation; it never sees the hidden test set.

- **AliCCP is the required benchmark** and determines 100% of the Primary metric score. **KuaiRand is a bonus benchmark**: a strong result on KuaiRand earns additional bonus points on top of the Primary metric score, but skipping it does not reduce the AliCCP-based score.
- Per-dataset metrics: **KuaiRand** → NDCG@K / Recall@K; **AliCCP** → CTR AUC / CVR AUC. Within each dataset, the score is the **equal-weighted average of each metric's *absolute* improvement over the official baseline** on the hidden test set. For every metric *m*:

  Code block

  ```
  delta(m) = score_agent(m) - score_baseline(m)
  score_dataset = mean over m of  delta(m)
  ```

Robustness. Not judged by whether the agent ever hits a failure, but by **how it handles one** — recovering, retrying, or routing around a failed step (a code error, a timeout, an unexpected input) so that long iterative runs neither crash, stall, nor diverge before hitting the compute/wall-clock budget.

Innovation & Problem Insight

Judged on what the agent identified as worth trying and why — not on implementation.

- What the agent chose to target across the full algorithmic stack (features, model architecture, training strategy, evaluation loop, etc. — improvements are not limited to the model itself) and the reasoning behind that choice.
- Originality in drawing on published methods, papers, or public solutions — rewarding agents that go beyond naive baseline tweaks.

Impact & Relevance — Autonomy

Autonomy. How much of the improvement loop the agent drives on its own — proposing and testing changes based on its own evaluation of results, not just tuning the model architecture. Measured primarily by the **number of manual interventions** required to reach the converged result; fewer interventions score higher, with fully autonomous runs scoring highest. The fewer humans required, the more this reflects real acceleration of recommender-system R&D.

Feasibility & Practicality — Resource Consumption

How much it costs — in both LLM usage and GPU compute time — to reach the converged result.

- **Token consumption.** Total input + output tokens used by the agent's LLM calls across the run.
- **GPU time.** Total GPU-hours consumed during training and evaluation to reach the converged result — captures the actual compute resources used in a way that wall-clock time alone cannot (e.g. running on more GPUs in parallel looks fast on the clock but is not necessarily cheaper).

2.7 References

[1] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry, "MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering," OpenAI, 2024. arXiv:2410.07095. https://doi.org/10.48550/arXiv.2410.07095

[2] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-Driven Exploration in the Space of Code," 2025. arXiv:2502.13138. https://doi.org/10.48550/arXiv.2502.13138

[3] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, "The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search," 2025. arXiv:2504.08066. https://doi.org/10.48550/arXiv.2504.08066

2.8 Appendix A. A Primer on Recommender Systems

> This appendix gives participants without a recommender-systems background just enough to get started. It is a concept map plus an annotated reading list — not a textbook. Use it to understand the two benchmarks and to know what to look up when you get stuck.

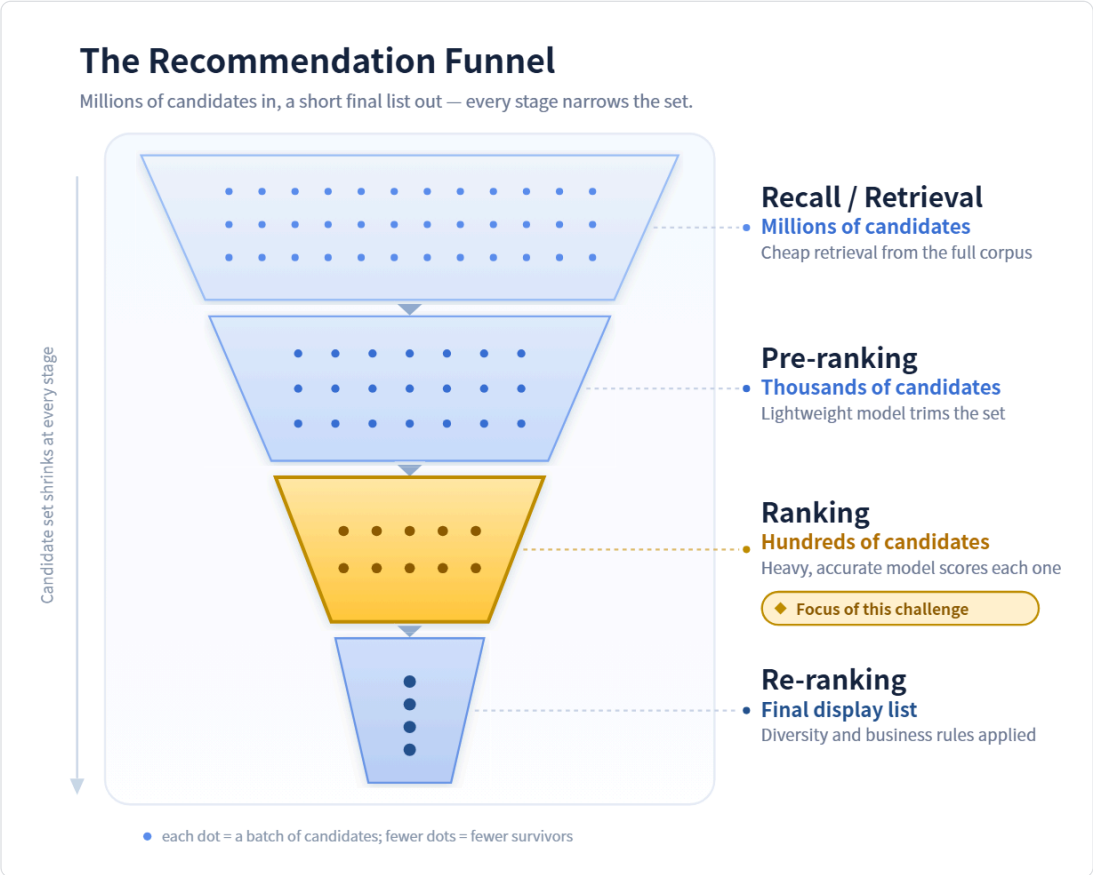
A.1 The Big Picture: The Recommendation Pipeline

A modern industrial recommender does not score every item directly. It runs a funnel of stages, each narrowing the candidate set:

```
Recall  →  Pre-ranking  →  Ranking  →  Re-ranking
millions    thousands       hundreds     final list
```

- **Recall / Retrieval**: cheaply retrieve a few thousand candidates from millions.
- **Pre-ranking**: a lightweight model trims the candidates further.
- **Ranking**: a heavy, accurate model scores each candidate. **This challenge mostly lives here.**
- **Reranking**: adjust the final ordering for diversity, business rules, and so on.

> For this competition you mainly need the **ranking** stage. The two datasets are ranking/prediction tasks, not full end-to-end pipelines.



A.2 Core Tasks: CTR, CVR, and the Funnel

Most industrial ranking is framed as predicting the probability of user feedback:

- **CTR (Click-Through Rate)** — `P(click | impression)`. The user saw the item; will they click?

- **CVR (Conversion Rate)** — `P(conversion | click)`. The user clicked; will they convert (buy)?
- **The funnel**: `impression → click → conversion`. Because these stages are linked, two well-known problems arise:
  - **Sample selection bias**: CVR is only observed on *clicked* items, yet must be predicted for *all* impressions.
  - **Data sparsity**: conversions are far rarer than clicks.

> **AliCCP** is exactly this CTR × CVR funnel. The ESMM model (see A.7.2) was designed to solve the two problems above, and was published on this dataset.

A.3 Multi-Task & Multi-Feedback Learning

Real users produce many signals (click, like, follow, comment, watch-time, and so on). Predicting them jointly — rather than training a separate model per signal — shares representations and tends to improve every task.

- Why it matters here: **KuaiRand** provides **12 feedback signals**; **AliCCP** provides the CTR + CVR pair.
- The key idea is to balance *shared* parameters (which transfer useful knowledge across tasks) against *task-specific* parameters (which prevent conflicting tasks from hurting one another — the "seesaw" problem).

A.4 Evaluation Metrics

Metric	Intuition	Used for
AUC	Probability that a random positive is ranked above a random negative. Threshold-free and robust to class imbalance.	CTR / CVR (AliCCP)
NDCG	Quality of a *ranked list*, rewarding relevant items near the top (with a position discount).	Ranking quality (KuaiRand)
Recall	Fraction of all relevant items that appear in the returned list.	Coverage (KuaiRand)

> **Offline vs. online**: a higher offline metric does not always mean better real-world performance (because of distribution shift and feedback loops). This competition is evaluated offline, but it is worth knowing the gap exists.

A.5 Feature Engineering Basics

- **ID features**: user ID, item ID, category ID — high-cardinality discrete features.
- **Embedding**: map each discrete ID to a learnable dense vector. This is the foundation of all deep recommenders.
- **Feature crossing**: combine features (e.g. user × category) to capture interactions. Models such as FM and DeepFM automate this.

A.6 Annotated Reading List

> 💡 Tip: If you find reading the following material challenging or find you have missing background knowledge, you can use AI assistants to explain it to you.

The goal here is only to understand **how a recommender system is structured** — the recall → ranking → re-ranking pipeline — and where the ranking stage (which this challenge targets) sits within it. You do **not** need to read a whole course; the introductory overview is enough. **Read just one of the following:**

- Google, *Recommendation Systems* (Machine Learning Crash Course), the **Overview** section — `https://developers.google.com/machine-learning/recommendation` A short, official overview of the pipeline. Note: Google calls the ranking stage **"scoring"** — this is the same thing as **ranking**, and it is the part this challenge focuses on.
- Wang Shusen, *Recommender Systems*, **Chapter 1 (Overview)** — `https://github.com/wangshusen/RecommenderSystem` The most beginner-friendly Chinese resource; the first chapter alone gives the full architecture.

3. Implement a GPU Kernel for a Transformer Layer

✨ **Technical Workshop Webinar with Q&A** will be held on **28 Aug, 3:00 to 3:45pm.**

Click here to join the webinar!

3.1 Background

Transformer is a widely used neural network architecture in modern AI. It is the core structure behind many natural language processing, computer vision, speech, recommendation, and large language model systems.

The main idea of Transformer is **self-attention**. Self-attention allows each token in a sequence to interact with other tokens directly. Compared with recurrent models, Transformer can process tokens in parallel, which makes it suitable for GPU acceleration.

Given an input sequence represented as a matrix:

$$X \in \mathbb{R}^{N \times d}$$

where N is the sequence length and d is the hidden dimension, the Transformer first projects the input into Query, Key, and Value matrices:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

The scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of each attention head. The scaling factor $\sqrt{d_k}$ is used to prevent the dot-product values from becoming too large, which could make the softmax distribution unstable.

However, the computation of Transformer is expensive. Important operations include matrix multiplication, attention score calculation, softmax, normalization, and feed-forward layers. These operations may be limited by GPU compute throughput, memory bandwidth, cache efficiency, kernel launch overhead, and tensor core utilization.

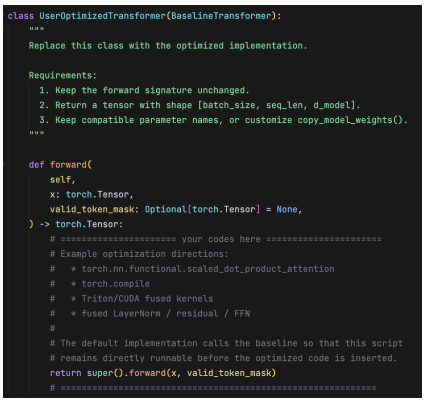
In this competition, participants are asked to use AI-assisted methods to optimize the runtime efficiency of a Transformer structure on a given GPU model. The optimized implementation should improve performance while keeping the output numerically correct compared with the reference implementation.

Participants may consider optimization methods such as operator fusion, memory layout optimization, reduced-precision computation, tensor core usage, softmax optimization, and custom CUDA, Triton, TensorFlow, or PyTorch implementations.

The goal of this task is to explore how AI can help developers analyze Transformer workloads, identify bottlenecks, and generate more efficient implementations for specific GPU hardware.

3.2 Problem Statement

- Given a fixed formula of transformer layer, participants need to submit one or several GPU kernels that implement the layers that can pass the given test cases.
- The test cases would be written in pytorch or tensorflow and the participants can modify the layer implementation if they need, which means they can decide which parts of the layers should be fused into 1 kernel.
- The test case would compare the differences between the implementation of participants and the original pytorch/tensorflow implementation, the diff should be small enough (relative error < 0.02, abs error < 0.002).
- The test cases would contain different shapes of input, including large/small batchsize, large/small sequence length, large/small dimensions, etc. The participants can choose different implementations for different shapes by adding shape checks in the implementation of layers. All the combinations of input shapes will be told to the participants.
- The use of AI tools is encouraged so that the participants can implement different kernels for different input shapes in limited time.
- Optimize & test your codes on your own machine. Different methods may be used to optimize the codes depending on the machine (GPU cards) you use.
- Provide a clear tech report including details on the AI skills/tools used to get bonus points.
- What participants need to do:
  - Download the benchmark scripts (choose either torch or tensorflow, one of them would be enough).
  - Implement the customized-implementation part and optimize it as fast as you can by AI or by hand.



```python
class UserOptimizedTransformer(BaselineTransformer):
    """
    Replace this class with the optimized implementation.

    Requirements:
      1. Keep the forward signature unchanged.
      2. Return a tensor with shape [batch_size, seq_len, d_model].
      3. Keep compatible parameter names, or customize copy_model_weights().
    """

    def forward(
        self,
        x: torch.Tensor,
        valid_token_mask: Optional[torch.Tensor] = None,
    ) -> torch.Tensor:
        # ====================== your codes here ======================
        # Example optimization directions:
        #   * torch.nn.functional.scaled_dot_product_attention
        #   * torch.compile
        #   * Triton/CUDA fused kernels
        #   * fused LayerNorm / residual / FFN
        #
        # The default implementation calls the baseline so that this script
        # remains directly runnable before the optimized code is inserted.
        return super().forward(x, valid_token_mask)
        # ============================================================
```

- Run the script on your own machine.
- Provide a clear tech report illustrating what the environment is (CPU, GPU, DISK, etc), what kind of optimizations you have done, and the final test results.

3.3 Constraints & Scope

Category	Constraints & Scope Details
In scope	AI-based code generation, GPU kernel fusion, profile tools usage, etc.
Out of scope	Production-ready deployment.

3.4 Available Resources / Data

You can download 1 of these, and run it on your own machine:

Torch Benchmark script	Tensorflow Benchmark script
torch_transformer_benchmark.py	tensorflow_transformer_benchmark.py

3.5 Deliverables

1. **Written Project Description (via Devpost)**

- Provide a clear written description of your project that includes:
  - How your solution addresses the problem statement
  - Development tools used (e.g. VSCode, Colab, Jupyter)
  - APIs used (e.g. OpenAI GPT-4o, Google Maps API)
  - Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
  - Datasets and assets used (e.g. Google Local Reviews dataset, manually labelled data)

2. **Public Code/GitHub Repository**

- Submit a link to a public Code/GitHub repository containing:
  - Well-structured, commented code covering all components of your solution
  - A README file that includes:
    - Project overview
    - Setup and installation instructions
    - Steps to reproduce your results
    - A brief reflection on your solution's limitations and what you would improve given more time
    - Team member contributions (if applicable)

3. **Demo Video**

Submit a short video that:

- Demonstrates your solution working end-to-end (e.g. inference results, dashboard, model predictions)

- Is uploaded to YouTube and set to public visibility
- Is linked in your Devpost description
- Does not include third-party trademarks or copyrighted content without permission

Note for backend/NLP tracks: If a front-end interface is not applicable to your solution, a walkthrough video showing API usage, inference examples, or result analysis is accepted.

3.6 Judging Criteria

Judging Criteria	Definition	Weight
Technical Execution	The solution demonstrates strong engineering fundamentals, such as well-structured code, thoughtful architecture, and effective use of APIs or models. The demo runs reliably, and the technical complexity reflects deliberate, capable decision-making.	**35%**
Innovation & Problem Insight	The project demonstrates originality in both idea and approach. It stands out for the sharpness of its problem understanding — how clearly the team has framed the challenge, why it matters, and how directly the solution addresses it.	**20%**
Impact & Relevance	The project has clear potential to deliver value to real users or stakeholders — with meaningful reach, tangible benefit, and relevance that goes beyond solving for the hackathon prompt alone.	**20%**
Feasibility & Practicality	The solution is realistic and buildable beyond a prototype. The approach is technically and operationally sustainable — resource usage is proportionate, the architecture holds under real-world conditions, and the implementation is grounded rather than speculative.	**15%**
Presentation & Communication	[Final Event Only]: The team communicates their work with clarity. The pitch tells a coherent story; from problem to solution to potential, and the team is able to respond to questions with depth, demonstrating genuine understanding of their own project.	**10%**

4. Shopping Copilot: AI Conversational Search and Recommendations

✨ **Technical Workshop Webinar with Q&A** will be held on **28 Aug, 4:00 to 4:45pm.**

Click here to join the webinar!

4.1 Background

Traditional e-commerce search engines heavily rely on static keyword matching, failing to capture the fluid shifts of genuine consumer psychology and the distinction between open-ended browsing and high-intent buying. In modern conversational commerce, constructing an intelligent agent that leverages dynamic context programming is critical to bridging the gap between ambiguous user queries and complex product catalogs. Solving this challenge directly impacts core industrial metrics.

4.2 Problem Statement

Participants are challenged to architect an intelligent, next-generation shopping agent capable of navigating real-world customer dynamics. Moving beyond rigid search filters, the engineered system must demonstrate deep cognitive understanding, runtime architectural agility, and commercial efficiency using the provided Amazon dataset.

Specifically, the system should be built upon the following four core pillars:

I. Core Architecture: Intent Routing & Hybrid Pipeline

- **Dual-Track Routing**: Instantly detect the user's underlying intent—triggering a high-precision filter track for targeted **"Buying"** to lock hard constraints, and a diverse dense retrieval track for open-ended **"Browsing"** to unlock cross-category scenario matching.
- **Pipeline Base**: Construct an in-memory data stream featuring **"Multi-Route Retrieval → LLM Semantic Ranking"** (combining keyword, category, and vector similarity).

II. Dialog Strategy: Multi-Turn Scenario Evolution

- **Dynamic State Machine**: Build a robust conversational state tracker to gracefully handle dynamic **Information Accumulation** (incremental slots) and abrupt **Intent Override** (slot erasure and rewriting).
- **Proactive Guidance**: Trigger an immediate retrieval cutoff when facing **Over-Generality** (candidate pool overload) to actively generate structured, proactive clarification prompts that guide user convergence.

III. Self-Evolution: Dynamic Context Programming

- **Runtime Adaptation**: Leverage accumulated dialog history to perform **Personalized Context Distillation**, continuously updating short-term session states and long-term user profiles.
- **Adaptive Orchestration**: Utilize dynamic Context Programming to achieve runtime workflow re-orchestration and strategy alignment, ensuring the agent iteratively refines its own guidance logic.

IV. Evaluation Matrix: Product & Efficiency Metrics

Anchored on the final purchased record within the Amazon dataset, performance is quantified across three dimensions:

- **Coverage (Hit Rate@K)**: Measures the catalog recall and boundary capability during the retrieval stage.
- **Precision (MRR / Top-K Hit Rate)**: Evaluates the LLM's accuracy in pushing the exact purchased item to the absolute top of the recommendation list.
- **Efficiency (MTTC - Mean Turns to Conversion)**: Heavy rewards systems that guide the user to the correct product in fewer interaction rounds, penalizing unnecessary conversational cognitive load.

4.3 Constraints & Scope

Category	Constraints & Scope Details
In scope	Designing highly sensitive intent-detection modules to split traffic into "Buying" and "Browsing" tracks. Implementing heterogeneous retrieval routing (weights, custom dynamic truncation, and slot decay over time). Engineering runtime-adaptive memory layers for personalized context distillation. Fine-tuning prompt strategies or local scoring logic for the LLM ranking stage to compress decision paths.
Out of scope	UI/UX Development (evaluated purely via automated backend APIs and headless pipelines). Training or full-parameter fine-tuning of base foundational LLMs. Deploying heavy external industrial vector DB clusters (must run entirely in-memory for light execution). Multi-Modal Processing (restricted strictly to text catalogs, structured metadata, and text dialogs).
Limits	**Max Turns**: Hard limit of **10 turns** per session (forced termination and zero score if exceeded). **Catalog Mutation**: The Amazon product dataset is strictly read-only; no structural mutations or mock ASIN injections are allowed.
Allowed assumptions	Inputs are pre-cleaned text strings (no need to account for spelling correction, typos, or ASR noise). Product catalog, pricing, and category trees are static for the duration of the hackathon. Each session is simulated as an isolated single-user interaction (no multi-user concurrency stress needed).

4.4 Available Resources & Data

Participants receive a frozen and reproducible competition kit derived from the Amazon Reviews 2023 dataset.

Competition Data

- A frozen catalog containing 50,000 products from the Amazon Reviews 2023 `Clothing_Shoes_and_Jewelry` category.
- 200 labeled public development sessions for local testing and iteration.
- 800 additional sessions retained privately by the organizer for final evaluation.
- Public and private evaluation sessions use separate users and target products.

Participant Resources

- A weak BM25 starter Agent implemented in Python.
- A deterministic local evaluator for Hit Rate@10, MRR, MTTC, Efficiency, and the combined TechnicalScore.
- A published Python Agent interface and machine-readable API contract.
- Evaluation configuration, reproducible baseline results, data documentation, and submission rules.
- A SHA256 checksum file for verifying the downloaded catalog.

Participants can modify or replace the starter Agent while continuing to use the official local evaluator. The participant kit supports keyword retrieval, rule-based methods, dense retrieval, hybrid retrieval, reranking, local models, and external model APIs.

The organizer does not provide hosted model access, API keys, model tokens, or third-party API credits. A paid LLM is not required to complete the challenge. Teams that choose to use external services are responsible for their own credentials, usage limits, and costs, and must not publish secrets in their repositories.

Resources

- Participant repository:
  https://github.com/TechJam2026/techjam-conversational-search
- Participant Kit Release:
  https://github.com/TechJam2026/techjam-conversational-search/releases/tag/participant-kit
- Original data source and documentation:
  https://amazon-reviews-2023.github.io/

The competition catalog and evaluation sessions are prepared and frozen by the organizer. Participants do not need to download or reconstruct the full upstream Amazon Reviews 2023 dataset.

4.5 Deliverables

1. **Written Project Description (via Devpost)**

- Provide a clear written description of your project that includes:
  - How your solution addresses the problem statement
  - Development tools used (e.g. VSCode, Colab, Jupyter)
  - APIs used (e.g. OpenAI GPT-4o, Google Maps API)
  - Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
  - Datasets and assets used (e.g. Google Local Reviews dataset, manually labelled data)

2. **Public Code/GitHub Repository**

- Submit a link to a public Code/GitHub repository containing:
  - Well-structured, commented code covering all components of your solution
  - A README file that includes:
    - Project overview
    - Setup and installation instructions
    - Steps to reproduce your results
    - A brief reflection on your solution's limitations and what you would improve given more time
    - Team member contributions (if applicable, i.e. team participants, non-solo participants)

3. **Demo Video**

Submit a short video that:

- Demonstrates your solution working end-to-end (e.g. inference results, dashboard, model predictions)
- Is uploaded to YouTube and set to public visibility
- Is linked in your Devpost description
- Does not include third-party trademarks or copyrighted content without permission

Note for backend/NLP tracks: If a front-end interface is not applicable to your solution, a walkthrough video showing API usage, inference examples, or result analysis is accepted.

4.6 Judging Criteria

Judging Criteria	Definition	Weight
Technical Execution	The solution demonstrates strong engineering fundamentals, such as well-structured code, thoughtful architecture, and effective use of APIs or models. The demo runs reliably, and the technical complexity reflects deliberate, capable decision-making.	**35%**
Innovation & Problem Insight	The project demonstrates originality in both idea and approach. It stands out for the sharpness of its problem understanding — how clearly the team has framed the challenge, why it matters, and how directly the solution addresses it.	**20%**
Impact & Relevance	The project has clear potential to deliver value to real users or stakeholders — with meaningful reach, tangible benefit, and relevance that goes beyond solving for the hackathon prompt alone.	**20%**
Feasibility & Practicality	The solution is realistic and buildable beyond a prototype. The approach is technically and operationally sustainable — resource usage is proportionate, the architecture holds under real-world conditions, and the implementation is grounded rather than speculative.	**15%**
Presentation & Communication	[Final Event Only]: The team communicates their work with clarity. The pitch tells a coherent story; from problem to solution to potential, and the team is able to respond to questions with depth, demonstrating genuine understanding of their own project.	**10%**

5. Robust Detection of AI-Generated Images Under Real-World Transformations

✨ **Technical Workshop Webinar with Q&A** will be held on **28 Aug, 5:00 to 5:45pm.**

Click here to join the webinar!

5.1 Background

Generative AI tools are making it easier than ever to create highly realistic synthetic images at scale. This creates new risks for online platforms, including misinformation, impersonation, fraud, and reduced trust in digital content. In practice, detection becomes even harder after images are compressed, cropped, reposted, or lightly edited, so robust methods matter more than lab-only accuracy.

5.2 Problem Statement

We want participants to build a prototype that can distinguish AI-generated images from authentic images with strong robustness under realistic post-processing and redistribution scenarios. The goal is not only to achieve good detection performance on clean data, but also to maintain accuracy after transformations such as blur, compression, color adjustment, cropping, or rescaling. Solutions should present a clear technical approach, an evaluation strategy, and thoughtful discussion of trade-offs such as robustness, generalisation, and false positives.

Note: We consider robustness against a subset of the following augmentataions.

Transform	Parameters	Real-World Analog
JPEG Compression	quality = 90, 70, 50, 30	Social-media re-encode, messaging
Gaussian Blur	kernel σ = 0.5, 1.0, 2.0	Out-of-focus
Resize	scale 0.5× / 0.25× then upscale	Thumbnail generation
Gaussian Noise	σ = 0.02, 0.05, 0.10	Low-light sensor noise
Color Jitter	brightness/contrast/sat. ±20%	Filter apps, auto-enhance
Center Crop	crop 80%	Profile-picture cropping, framing

5.3 Constraints & Scope

Category	Constraints & Scope Details
In scope	Image-level AIGC detection, robustness to common image transformations, feature engineering, model design, evaluation design, error analysis, and explainability ideas
Out of scope	Full production deployment, platform-wide moderation systems, and non-image modalities such as video or audio
Limits	Assume a hackathon-scale prototype, limited compute, and no access to internal production systems. Teams should optimise for a convincing proof of concept rather than a production-grade service. **Note: Participants must use models with <2B parameters.**
Allowed assumptions	Teams may use public or properly licensed datasets, create their own transformed test cases, and make reasonable assumptions about deployment context as long as those assumptions are stated clearly.

5.4 Available Resources & Data

- Public or properly licensed image datasets for AIGC detection and image forensics.
- Self-created transformed samples using operations such as blur, compression, cropping, color adjustment, or rescaling.
- Public documentation for relevant machine learning and computer vision libraries.
- Datasets:
  - https://huggingface.co/datasets/saberzl/SID_Set
  - https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images
  - https://modelscope.cn/datasets/hy2628982280/WildFake/summary
    - For this modelscope dataset, please translate it via the translation button before use:

Validation Dataset (for Demonstration Purposes Only):

We choose **a subset of WildFake** for participants to demonstrate their models' performance and track iterative improvements. This dataset serves only as a reference benchmark and will not contribute to the final score. **Do not use the following data during training.** Specifically:

Dataset		# Num
Non-AIGC	COCO val2017	4998
AIGC	DALL·E Advanced	8843

5.5 Expected Deliverables

1. **Written Project Description (via Devpost)**

- Provide a clear written description of your project that includes:

- How your solution addresses the problem statement
- Development tools used (e.g. VSCode, Colab, Jupyter)
- Models or APIs used
- Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
- Datasets and assets used

2. **Public Code/GitHub Repository**

- Submit a link to a public Code/GitHub repository containing:
  - Well-structured, commented code covering all components of your solution
  - A script that takes an image directory as input and outputs a confidence score for each image, indicating the likelihood that it is AIGC-generated. The output should be a JSON file containing `image_path` and `pred` for each image.
  - A README file that includes:
    - Project overview
    - Setup and installation instructions
    - Steps to reproduce your results
    - A brief reflection on your solution's limitations and what you would improve given more time
    - Team member contributions (if applicable, i.e. team participants, non-solo participants)

3. **Demo Video**

- Submit a short video that:
  - Demonstrates your solution working end-to-end (e.g. inference results, dashboard, model predictions)
  - Is uploaded to YouTube and set to public visibility
  - Is linked in your Devpost description
  - Does not include third-party trademarks or copyrighted content without permission

4. **Robustness Evaluation Summary**

- Include a compact table or visual summary comparing performance on clean images versus transformed images.

5. **Error Analysis Note**

- Highlight representative false positives, false negatives, and any trade-offs in the proposed approach.

5.6 Judging Criteria

Judging Criteria	Definition	Weight
Technical Execution	The solution demonstrates strong engineering fundamentals, such as well-structured code, thoughtful architecture, and effective use of APIs or models. The demo runs reliably, and the technical complexity reflects deliberate, capable decision-making.	**35%**
Innovation & Problem Insight	The project demonstrates originality in both idea and approach. It stands out for the sharpness of its problem understanding — how clearly the team has framed the challenge, why it matters, and how directly the solution addresses it.	**20%**
Impact & Relevance	The project has clear potential to deliver value to real users or stakeholders — with meaningful reach, tangible benefit, and relevance that goes beyond solving for the hackathon prompt alone.	**20%**
Feasibility & Practicality	The solution is realistic and buildable beyond a prototype. The approach is technically and operationally sustainable — resource usage is proportionate, the architecture holds under real-world conditions, and the implementation is grounded rather than speculative.	**15%**

Presentation & Communication	The team communicates their work with clarity. [Final Event Only]: The pitch tells a coherent story; from problem to solution to potential, and the team is able to respond to questions with depth, demonstrating genuine understanding of their own project.	**10%**